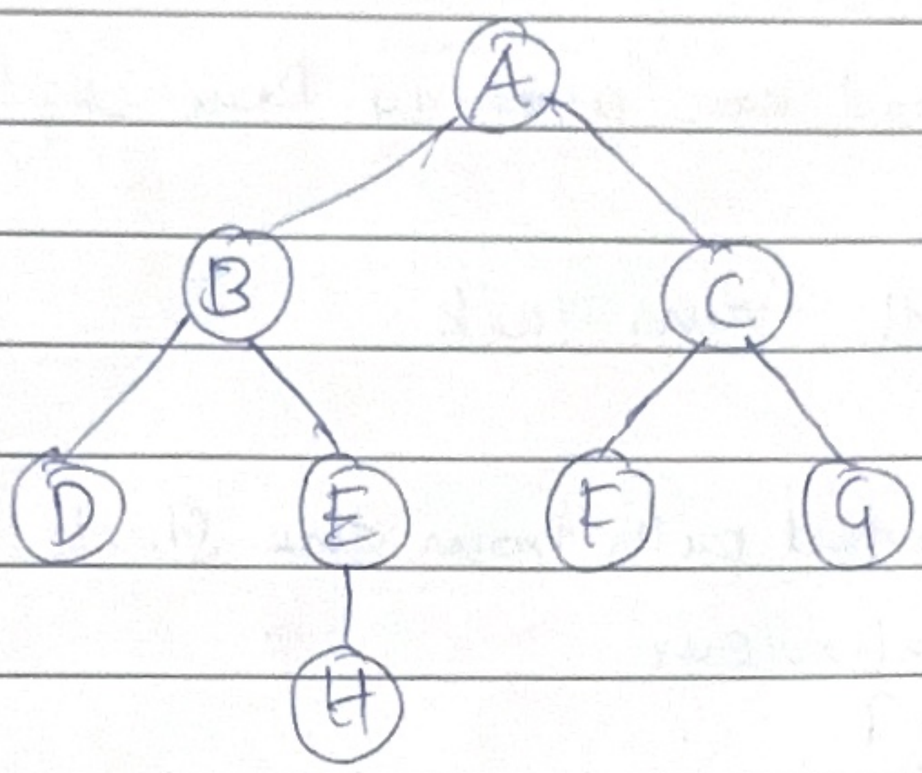
Lab-2b Depth-First Search

Find the shortest path from the start node (A) to a goal node (H) in a graph using DFS algorithm.



Step 1: Define the graph

```
graph = {'A': ['B', 'C'],
         'B': ['D', 'E'],
         'C': ['F', 'G'],
         'D': [],
         'E': ['H'],
         'F': [],
         'G': [],
         'H': []}
```

A → start node

H → goal node

Step 2: Define DFS function

```
def dfs(start, goal):
```

This function performs Depth-First Search to find a path from the start node to the goal node.

start → initial node

goal → destination node

Step 3: Initialize stack and visited list

stack = [[start]]

The stack stores complete paths instead of individual nodes.

Initially: stack = [['A']]

visited = [ ]

Stores the nodes that have already been explored.

Step 4: Remove path from stack

path = stack.pop()

Removes the last inserted path from the stack.

↳ follows LIFO behaviour.

Ex: stack = [A, B, C]

Remove C first

This is the main difference from BFS.

Step 5: Get current node

node = path[-1]

The last element of the path represents the current node being explored.

Ex: path = ['A', 'B', 'E']

node = E

Step 6: Check goal and mark visited

if node == goal:

return path, visited

If the current node is the goal node, the path is returned

if node not in visited:

visited.append(node)

If the node is not visited, it is added to the visited list.

Step 7: Add child nodes to stack

```
for n in reversed(graph[node]):
    stack.append(path+[n])
```

The child nodes of the current node are added to the stack. reversed() is used so that the left child is explored first.

Ex: A → B, C

Without reversing ⇒ C would be removed first

With reversing ⇒ B is placed on top and explored first.

Step 8: Execute DFS

```
p, v = dfs('A', 'H')
```

Start node = A

goal node = H

Step 9: Display result

```
print("DFS path:", p)
print("DFS visited:", v)
```

Prints:

The path from A to H

The order in which nodes were visited,

Output:

```
DFS path: ['A', 'B', 'E', 'H']
DFS visited: ['A', 'B', 'D', 'E']
```